Step 1: Create Tables from `superstore_raw`

Customers Table

```
CREATE TABLE customers AS
SELECT DISTINCT
    customer_id,
    customer_name,
    segment,
    country,
    city,
    state,
    postal_code,
    region
FROM superstore_raw;
```

Orders Table

```
CREATE TABLE orders AS
SELECT DISTINCT
    row_id,
    order_id,
    order_date,
    ship_date,
    ship_mode,
    customer_id,
    product_id,
    sales,
    quantity,
    discount,
    profit
FROM superstore_raw;
```

Products Table

```
CREATE TABLE products AS
SELECT DISTINCT
    product_id,
    category,
    sub_category,
    product_name
FROM superstore_raw;
```

Step 2: Required Queries

1. Orders with Sales Greater Than Average (Subquery)

```
SELECT *
FROM orders
WHERE sales >
(
    SELECT AVG(sales)
    FROM orders
);
```

2. Highest Sales Order for Each Customer (Subquery)

```
SELECT *
FROM orders o
WHERE sales =
(
    SELECT MAX(sales)
    FROM orders
    WHERE customer_id = o.customer_id
);
```

3. Total Sales for Each Customer (CTE)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_sales
ORDER BY total_sales DESC;
```

4. Customers with Above Average Sales (CTE + Subquery)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
```

```
)
SELECT *
FROM customer_sales
WHERE total_sales >
(
    SELECT AVG(total_sales)
    FROM customer_sales
);
```

5. Rank Customers Based on Total Sales (Window Function)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    total_sales,
    RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
FROM customer_sales;
```

6. Assign Row Numbers to Orders Within Each Customer

```
SELECT
    customer_id,
    order_id,
    sales,
    ROW_NUMBER() OVER
    (
        PARTITION BY customer_id
        ORDER BY sales DESC
    ) AS row_num
FROM orders;
```

7. Top 3 Customers Based on Total Sales

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
),
ranked_customers AS
```

```
(
    SELECT
        customer_id,
        total_sales,
        RANK() OVER (ORDER BY total_sales DESC) AS sales_rank
    FROM customer_sales
)
SELECT *
FROM ranked_customers
WHERE sales_rank <= 3;
```

Step 3: Final Combined Query

(Customer Name + Total Sales + Rank)

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT
    c.customer_name,
    cs.total_sales,
    RANK() OVER (ORDER BY cs.total_sales DESC) AS customer_rank
FROM customer_sales cs
JOIN customers c
ON cs.customer_id = c.customer_id
ORDER BY customer_rank;
```

Mini Project: Customer Sales Insights

Top 5 Customers

```
WITH customer_sales AS
(
    SELECT
        c.customer_name,
        SUM(o.sales) AS total_sales
    FROM customers c
    JOIN orders o
    ON c.customer_id = o.customer_id
    GROUP BY c.customer_name
)
SELECT *
FROM customer_sales
```

```
ORDER BY total_sales DESC
LIMIT 5;
```

Bottom 5 Customers

```
WITH customer_sales AS
(
    SELECT
        c.customer_name,
        SUM(o.sales) AS total_sales
    FROM customers c
    JOIN orders o
    ON c.customer_id = o.customer_id
    GROUP BY c.customer_name
)
SELECT *
FROM customer_sales
ORDER BY total_sales ASC
LIMIT 5;
```

Customers Who Made Only One Order

```
SELECT
    customer_id,
    COUNT(DISTINCT order_id) AS total_orders
FROM orders
GROUP BY customer_id
HAVING COUNT(DISTINCT order_id) = 1;
```

Customers with Above Average Sales

```
WITH customer_sales AS
(
    SELECT
        customer_id,
        SUM(sales) AS total_sales
    FROM orders
    GROUP BY customer_id
)
SELECT *
FROM customer_sales
WHERE total_sales >
(
    SELECT AVG(total_sales)
    FROM customer_sales
);
```

Highest Order Value Per Customer

```
SELECT
    customer_id,
    MAX(sales) AS highest_order_value
FROM orders
GROUP BY customer_id
ORDER BY highest_order_value DESC;
```

Brief Insights (for Report)

1. Top customers contribute a large share of total revenue.
2. Bottom customers generate very little sales and may need targeted marketing.
3. Single-order customers indicate opportunities for customer retention.
4. Above-average customers are the most valuable customer segment.
5. Customer ranking helps identify high-performing customers for business decisions.